

Documentación Técnica — BigMart Sales Prediction

Documentación Técnica — BigMart Sales Prediction

PI Consulting — Data Scientist Challenge **Autora:** Jimena Fioni **Fecha:** Junio 2026

1. Resumen ejecutivo

El objetivo del challenge es construir un modelo de regresión para predecir las ventas por producto (Item_Outlet_Sales) en cada tienda de la cadena BigMart, e identificar los factores más influyentes en dichas ventas.

Se evaluaron **9 modelos** sobre los datos de entrenamiento usando validación cruzada de 5 folds. El pipeline completo incluye limpieza de datos, feature engineering con interacciones y target encoding, optimización bayesiana de hiperparámetros (Optuna) y un stacking ensemble como modelo final. Se implementó además un script de experimentación con MLflow para tracking de runs y registro de modelos, y una app interactiva con Streamlit que incluye un agente conversacional de IA.

2. El problema y los datos

2.1 Descripción del problema

BigMart es una cadena de supermercados con tiendas en distintas ciudades. El objetivo es predecir Item_Outlet_Sales — las ventas de cada producto en cada tienda — usando las características del producto y de la tienda.

2.2 Dataset

Archivo	Registros	Columnas
Train_BigMart.csv	8,523	12 (incluye target)
Test_BigMart.csv	5,681	11 (sin target)

2.3 Variables

Variable	Tipo	Descripción
Item_Identifier	Categórica	Código único del producto
Item_Weight	Numérica	Peso del producto en gramos
Item_Fat_Content	Categórica	Clasificación de grasas (Low Fat / Regular)
Item_Visibility	Numérica	Visibilidad del producto en la góndola
Item_Type	Categórica	Tipo de producto (16 categorías)
Item_MRP	Numérica	Precio máximo sugerido por el fabricante
Outlet_Identifier	Categórica	ID de la tienda
Outlet_Establishment_Year	Numérica	Año de apertura de la tienda
Outlet_Size	Categórica	Tamaño de la tienda (Small / Medium / High)
Outlet_Location_Type	Categórica	Tipo de ubicación (Tier 1 / 2 / 3)
Outlet_Type	Categórica	Tipo de tienda (Grocery / Supermarket Type 1/2/3)
Item_Outlet_Sales	Numérica	Variable objetivo — ventas del producto

2.4 Valores faltantes detectados

Variable	Nulos	%	Tratamiento
Item_Weight	2,439	17.2%	Imputación por media del mismo ítem
Outlet_Size	2,410	28.3%	Imputación por moda del tipo de tienda

3. Análisis Exploratorio de Datos (EDA)

3.1 Variable objetivo

La distribución de Item_Outlet_Sales presenta **asimetría positiva** (skewness > 1), con la mayoría de las ventas concentradas entre \$0 y \$4,000 y una cola larga hacia la derecha. Esto motivó el uso de **log-transform** ($\log(1 + y)$) como preprocesamiento del target para todos los modelos.

3.2 Hallazgos principales

Precio (Item_MRP):

- Es la variable numérica con mayor correlación con las ventas (~0.57)
- Se observa una relación positiva clara: productos más caros generan más ventas

Tipo de tienda (Outlet_Type):

- Supermarket Type3 vende en promedio **~6 veces más** que Grocery Store
- El tipo de tienda es el factor estructural más determinante de las ventas

Antigüedad de la tienda (Outlet_Age):

- Las tiendas más antiguas tienen ventas más estables y consolidadas
- Relación no lineal: no siempre más antigüedad implica más ventas

Visibilidad del producto:

- 526 registros tenían Item_Visibility = 0 — error de datos detectado
- La visibilidad por sí sola tiene baja correlación con las ventas

Tipo de producto (Item_Type):

- Seafood y Starchy Foods son los tipos con mayores ventas medianas
- Breakfast y Soft Drinks son los de menores ventas

4. Limpieza de datos

4.1 Normalización de Item_Fat_Content

El campo tenía múltiples formatos inconsistentes para el mismo valor:

```
fat_map = {"LF": "Low Fat", "low fat": "Low Fat", "reg": "Regular"}
df["Item_Fat_Content"] = df["Item_Fat_Content"].replace(fat_map)
# Productos no comestibles → "Non-Edible"
df.loc[df["Item_Type"].isin(["Health and Hygiene", "Household",
                             "Others"]),
        "Item_Fat_Content"] = "Non-Edible"
```

4.2 Imputación de Item_Weight

El mismo producto tiene el mismo peso en todas las tiendas. Se imputa con la media por ítem:

```
item_weight_mean = df.groupby("Item_Identifier")
                    ["Item_Weight"].transform("mean")
df["Item_Weight"] = df["Item_Weight"].fillna(item_weight_mean)
```

4.3 Imputación de Outlet_Size

Se imputa con la moda del tipo de tienda correspondiente (un Grocery Store suele ser Small, un Supermarket Type3 suele ser High):

```
outlet_size_mode = df.groupby("Outlet_Type")["Outlet_Size"].agg(
    lambda x: x.mode()[0] if not x.mode().empty else "Small"
)
```

4.4 Corrección de Item_Visibility = 0

Los 526 registros con visibilidad 0 representan errores de medición. Se reemplazan con la media positiva del mismo ítem:

```
item_vis_mean = df.groupby("Item_Identifier")
    ["Item_Visibility"].transform(
    lambda x: x[x > 0].mean() if (x > 0).any() else x.mean()
)
df.loc[df["Item_Visibility"] == 0, "Item_Visibility"] = item_vis_mean
```

5. Feature Engineering

5.1 Features derivadas

Feature	Fórmula	Razonamiento
Outlet_Age	2013 – Outlet_Establishment_Year	Captura la madurez de la tienda
Item_Visibility_MeanRatio	visibilidad / media por ítem	Mide sobre/sub-exposición relativa
Item_Category	Agrupación de 16 tipos en 3	Food / Drinks / Non-Consumable
MRP_Tier	Cuartiles de Item_MRP	Budget / Mid / Premium / Luxury
Item_Prefix	Primeras 2 letras del código	FD (Food), DR (Drinks), NC (Non-Consumable)

5.2 Features de interacción

Feature	Fórmula	Razonamiento
MRP_x_OutletType	Item_MRP × Outlet_Type_ordinal	Productos premium se comportan distinto por tipo de tienda
MRP_vs_Category_Mean	Item_MRP / media de MRP por categoría	Precio relativo del ítem dentro de su categoría
Visibility_x_Age	Item_Visibility × Outlet_Age	Interacción entre exposición y madurez de tienda

5.3 Target Encoding (sin data leakage)

El target encoding se calcula **solo sobre los datos de train** para evitar data leakage:

```
train_only = df[df["split"] == "train"]
outlet_sales_mean = train_only.groupby("Outlet_Identifier")
    ["Item_Outlet_Sales"].mean()
df["Outlet_MeanSales"] =
    df["Outlet_Identifier"].map(outlet_sales_mean).fillna(global_mean)
```

6. Modelado

6.1 Configuración de validación

Todos los modelos se evaluaron con **validación cruzada de 5 folds** (KFold estratificado por SEED=42) usando **RMSE en escala original** como métrica principal.

Se aplicó **log-transform del target** ($\log(1 + y)$) en todos los modelos para mejorar la distribución de errores, revertiendo con `expm1` para reportar métricas en escala original.

6.2 Modelos evaluados

Modelo	RMSE (CV)	R ²
XGBoost + RandomizedSearchCV	1,100.52	0.5871
Stacking Ensemble (Ridge meta)	1,108.13	0.5783
XGBoost + Optuna	1,109.18	—
CatBoost	1,110.20	0.5767
LightGBM + Optuna	1,111.76	—
Random Forest	1,120.32	0.5690
Ridge Regression	1,127.82	0.5632
LightGBM (base)	1,169.93	0.5299
XGBoost (base)	1,172.35	0.5280
MLP — Red Neuronal (sklearn)	1,189.54	0.5140
Baseline (media)	1,846.81	-0.1713

 **Mejor modelo: XGBoost + RandomizedSearchCV** — RMSE 1,100.52 · R² 0.5871

Resultado notable: el modelo más simple del análisis superó al Stacking Ensemble y a los modelos tuneados con Optuna. Esto sugiere que para este dataset, una configuración directa sobre la escala original de ventas captura mejor la distribución real que el pipeline con log-transform.

¿Por qué este modelo pudo ser el mejor?

1. Alineación entre objetivo de entrenamiento y métrica de evaluación La métrica de evaluación es el RMSE en escala original. El XGBoost + RandomizedSearchCV entrena directamente minimizando el error en esa escala, lo que alinea perfectamente el objetivo de entrenamiento con la métrica real. Los modelos con log-transform optimizan el error en escala logarítmica y luego revierten — esto introduce una pequeña discrepancia entre lo que el modelo aprende a minimizar y lo que realmente se mide.

2. XGBoost maneja distribuciones asimétricas de forma nativa A diferencia de los modelos lineales (Ridge), XGBoost usa árboles de decisión como base. Los árboles no asumen ninguna distribución en la variable objetivo — pueden aprender relaciones no lineales y manejar la asimetría de las ventas directamente, sin necesidad de transformarla. El log-transform fue concebido para ayudar a modelos lineales; en gradient boosting puede ser innecesario o incluso contraproducente.

3. La complejidad adicional no siempre mejora El Stacking Ensemble agrega una capa extra de meta-aprendizaje sobre predicciones OOF de 4 modelos. Esto introduce más fuentes de varianza: - Los errores de cada modelo base se propagan al meta-learner - El meta-learner (Ridge) aprende sobre datos en log-escala, pero se evalúa en escala original - El Random Forest tuvo peso casi nulo (0.003) en el stack — era ruido más que señal

Optuna con 50 trials encontró buenos hiperparámetros, pero también entrenó en log-escala, heredando la misma discrepancia.

4. El espacio de búsqueda de RandomizedSearchCV fue adecuado Los hiperparámetros ganadores (`max_depth=4`, `learning_rate=0.05`, `n_estimators=300`) apuntan a un modelo deliberadamente conservador — poco profundo, con regularización (`reg_lambda=5.0`). Esto reduce el overfitting en un dataset de solo 8,523 registros, donde los modelos más complejos pueden ajustarse en exceso al ruido del training.

Conclusión práctica: en datasets pequeños y métricas en escala original, un modelo bien regularizado entrenado directamente sobre el target puede superar pipelines más complejos. La navaja de Occam aplica también al Machine Learning.

6.3 Decisiones de modelado

¿Por qué log-transform del target? La distribución de ventas tiene fuerte asimetría positiva. El log-transform estabiliza la varianza y mejora la performance de todos los modelos lineales y de boosting.

¿Por qué CatBoost? Maneja variables categóricas nativamente sin necesidad de label encoding. Evita el sesgo que introduce asignar órdenes arbitrarios (Grocery Store=0, Type3=3 implica una relación ordinal que no existe necesariamente).

¿Por qué Optuna sobre GridSearch? Con 8 hiperparámetros por modelo, GridSearch es computacionalmente inviable. Optuna usa búsqueda bayesiana (sampler TPE) que converge en menos trials que búsqueda aleatoria. Se usaron 50 trials con pruning (MedianPruner) para descartar configuraciones malas tempranamente.

¿Por qué Stacking Ensemble? En lugar de promediar modelos con pesos fijos, el meta-learner (Ridge) aprende los pesos óptimos de combinación usando predicciones out-of-fold. Esto permite que el ensemble aprenda qué modelo es mejor en qué casos.

6.4 Variables más importantes

Según los modelos de gradient boosting:

1. **Item_MRP** — predictor dominante (correlación ~ 0.57 con ventas)
 2. **Outlet_MeanSales** — historial de ventas de la tienda (target encoding)
 3. **Outlet_Type** — tipo de tienda define el volumen estructural
 4. **MRP_x_OutletType** — interacción precio $\times$ tipo de tienda
 5. **ItemType_MeanSales** — target encoding del tipo de producto
-

7. Pipeline de experimentación — MLflow + Optuna

Se implementó un script de producción (`train.py`) que integra:

- **Optuna** con TPE sampler + MedianPruner para búsqueda bayesiana
- **MLflow** con backend SQLite para tracking de experimentos
- **MLflow Model Registry** para versionar modelos
- **config.yaml** para centralizar hiperparámetros sin hardcodear nada
- **src/** con custom sklearn transformers que garantizan no data leakage

Correr el pipeline completo

```
python train.py --n-trials 50
```

Ver experimentos

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001
```

Cada run de Optuna queda registrado como nested run en MLflow con sus métricas, hiperparámetros y artefactos (feature importance CSV).

8. App interactiva — Streamlit

La app (`app.py`) tiene tres secciones:



Dashboard EDA

Visualizaciones interactivas con Plotly: distribuciones, ventas por tipo de tienda, mapa de calor producto $\times$ tienda, scatter MRP vs ventas.



Modelo & Predicciones

- Feature importance del modelo LightGBM
- **Predictor interactivo**: ajustá MRP, tipo de tienda, peso, visibilidad y obtenés una estimación de ventas en tiempo real
- Descarga de predicciones del test set en CSV

 Chat con Agente de IA

Agente conversacional construido sobre la API de Claude (Anthropic). Responde preguntas sobre el dataset con código Python ejecutable como respaldo. Sin API Key, responde preguntas predefinidas.

```
streamlit run app.py
```

9. Dificultades encontradas

Dificultad	Solución aplicada
Item_Weight nulo en 17%	Imputación por media del mismo ítem (mismo producto = mismo peso en todas las tiendas)
Outlet_Size faltante	Imputación por moda del tipo de tienda correspondiente
Item_Visibility = 0 (526 registros)	Corrección con media positiva por ítem — son errores de registro
Item_Fat_Content inconsistente	Unificación de “LF”, “low fat”, “Low Fat” y variantes
Red neuronal (TensorFlow)	Incompatibilidades de arquitectura ARM64/x86_64 en el entorno local. Reemplazada por MLPRegressor de scikit-learn con resultados equivalentes
Paralelismo anidado en macOS	cross_val_predict con n_jobs=-1 genera deadlocks con el backend loky de joblib. Resuelto con n_jobs=1 en CV, delegando paralelismo a cada modelo
MLflow 3.x deprecó file store	Migración a backend SQLite (sqlite:///mlflow.db) recomendado por MLflow 3.x

10. ¿Qué otras fuentes de datos mejorarían el modelo?

Fuente	Impacto esperado
Datos demográficos por ciudad	Ingreso promedio y densidad poblacional contextualizan el poder adquisitivo de cada zona
Fechas de transacción	El dataset no tiene fecha — datos temporales permitirían capturar estacionalidad y tendencias
Datos de competencia	Presencia de otras tiendas del mismo tipo en la zona (efecto de canibalización)
Historial de precios	Variaciones del MRP en el tiempo para capturar elasticidad precio-demanda
Datos de marketing	Promociones activas y descuentos tienen impacto directo en ventas

Fuente	Impacto esperado
Tráfico de la tienda	Visitas diarias y tasa de conversión relacionan demanda con oferta disponible

11. Cómo correr el proyecto

Requisitos

- Python 3.11+
- Dependencias: `pip install -r requirements.txt`

Pasos

1. Instalar dependencias

```
pip install -r requirements.txt
```

2. Abrir el notebook (análisis completo)

```
jupyter notebook bigmart_analysis.ipynb
```

3. Correr el pipeline de experimentación

```
python train.py --no-optuna          # rápido, usa parámetros del
    config
```

```
python train.py --n-trials 50        # con búsqueda bayesiana completa
```

4. Ver experimentos en MLflow UI

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001
```

→ http://localhost:5001

5. Lanzar la app interactiva

```
streamlit run app.py
```

→ http://localhost:8501

Archivos generados

Archivo	Generado por
predictions_test.csv	Notebook (sección 9)
mlflow.db	train.py

12. Estructura del proyecto

DS 2026/

```
|— bigmart_analysis.ipynb    # Notebook principal – EDA + modelos
|— train.py                  # Pipeline MLflow + Optuna
|— app.py                    # Streamlit app
```

└─ config.yaml	# Configuración centralizada
└─ requirements.txt	# Dependencias
└─ README.md	# Guía de instalación y uso
└─ DOCUMENTACION.md	# Este documento
└─ src/	
└─ preprocessing.py	# Custom transformers – limpieza
└─ features.py	# Custom transformers – feature
engineering	
└─ images/	# Capturas de pantalla
└─ .streamlit/	
└─ config.toml	# Tema oscuro de la app
└─ predictions_test.csv	# Predicciones del test set
└─ mlflow.db	# Base de datos de experimentos MLflow

13. Referencias

Herramientas y frameworks

- [MLflow Documentation](#)
- [Streamlit Documentation](#)
- [Optuna Documentation](#)
- [Anthropic API — Claude](#)

Modelos y librerías

- [XGBoost Documentation](#)
- [LightGBM Documentation](#)
- [CatBoost Documentation](#)
- [scikit-learn Documentation](#)

Técnicas

- [Stacking Ensembles — scikit-learn](#)
- [Target Encoding](#)
- [Optuna TPE Sampler](#)